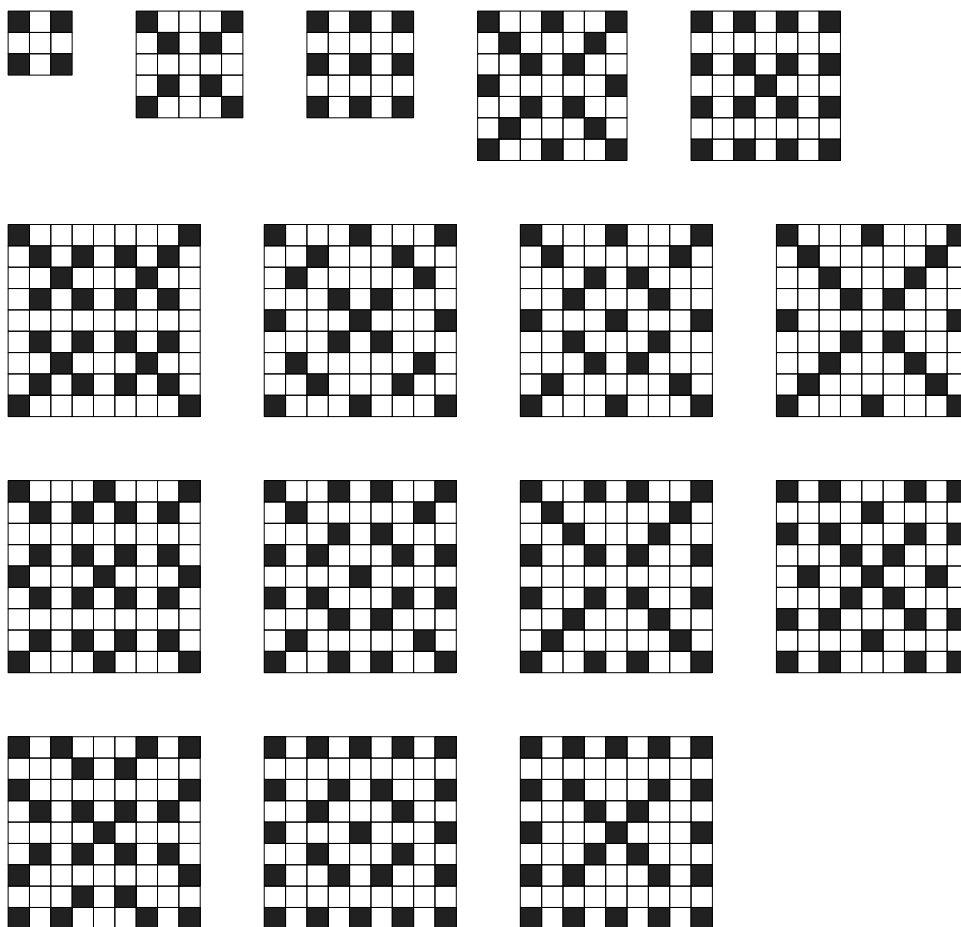


September 8, 2026 at 19:54

1. Introduction. Exercise 7.2.2.1–442 asks us to enumerate all hitori covers of $P_m \square P_n$ for $1 \leq m \leq n \leq 9$, and its answer prints a 9×9 matrix of totals together with several further statistics. This program computes all of them.

A *hitori cover* is defined in exercise 439: a set $U \subseteq V$ with (i) $G \mid U$ connected; (ii) if $v \notin U$ and $u - v$ then $u \in U$; (iii) if $u \in U$ and $v \in U$ for all $u - v$, then $G \mid (U \setminus u)$ is not connected. Answer 439 reads those three conditions back in the standard vocabulary: (ii) says U is a vertex cover, so (i) and (ii) say it is a *connected vertex cover*, and (iii) makes it a *minimal* one. In hitori terms U is the white cells: they hang together, no two black cells touch, and no further cell can be blackened.

2. Here are the covers that are symmetric all eight ways—one of size three, two of size five, two of size seven, and eleven of size nine. They are the rarest of the patterns in the gallery of answer 442, and this program finds them in the mode called **gallery**.



3. We work with the black cells $B = V \setminus U$ throughout, because that is what the answer's matrices count and what its pictures show. So B is independent, $V \setminus B$ is connected, and B is maximal with those two properties.

The answer says how it was done: a frontier-based algorithm builds a ZDD for the family of all B meeting the first two conditions, and then a subroutine that discards subsets leaves the maximal ones. We follow that plan exactly. The diagrams come from the **bdd** package (github.com/sjnam/bdd), a Go rendering of Knuth's BDD15; the frontier is ours.

4. The maximality step is two lines, and worth a word. Removing a black cell from B keeps B independent, and it hands one more cell to the white region—a cell all of whose neighbors are already white, since B was independent—so the white region stays connected. Hence the family is closed downwards, and a member fails to be maximal exactly when it is some member with one element removed. In ZDD algebra that family is $\bigcup_v f/v$, a union of quotients, and the maximal members are what is left when it is subtracted.

```
package main
import (
    "flag"
    "fmt"
    "math"
    "math/big"
    "strconv"
    "time"
    "github.com/sjnam/bdd"
)
< The frontier 7 >
< Bitboards 15 >
< Brute force 19 >
< Symmetry 21 >
< Reporting 29 >
func main() {
    < Read the command line 5 >
    < Do what the mode asks 6 >
}
```

5. The board size is a knob because the whole table takes a few seconds and the smaller ones are instant; `-n 6` is enough to watch every mode work.

< Read the command line 5 > $\equiv$

```
mode := flag.String("mode", "all", "counts,␣brute,␣sym,␣claims,␣two,␣gallery,␣or␣all")
size := flag.Int("n", 9, "the␣largest␣board␣to␣consider")
flag.Parse()
```

This code is used in section 4.

6. $\langle \text{Do what the mode asks } 6 \rangle \equiv$

```

N := *size
if *mode ≡ "counts" ∨ *mode ≡ "all" {
  counts(N)
}
if *mode ≡ "brute" ∨ *mode ≡ "all" {
  brute(min(N, 6))
}
if *mode ≡ "sym" ∨ *mode ≡ "all" {
  symmetry(N)
}
if *mode ≡ "claims" ∨ *mode ≡ "all" {
  claims(N)
}
if *mode ≡ "two" ∨ *mode ≡ "all" {
  twoByN()
}
if *mode ≡ "gallery" ∨ *mode ≡ "all" {
  gallery(N)
}

```

This code is used in section 4.

7. The frontier. Cells are numbered in row-major order and decided one at a time. When cell $i = rn + c$ comes up, the cells that still have undecided neighbors are the previous n , one per column: $comp[c]$ holds the cell directly above, and $comp[c - 1]$ the cell to the left. That is the frontier.

An entry is *black*, or else a small label naming the white component the cell belongs to. Labels are renumbered by first appearance before a state is used as a memo key, so that two states differing only in the names of their components are recognized as one.

⟨The frontier 7⟩ $\equiv$

```

const black = int8(0)
type state struct {
    comp [16]int8
    closed bool // a white component has already been completed
}
var (
    m, n int
    z *bdd.ZDD
    memo map[string]bdd.Func
    seen int // distinct (level, state) pairs: the unreduced diagram
)

```

This code is also defined in sections 8, 9, 13, 14.

This code is used in section 4.

8. $\langle \text{The frontier } 7 \rangle + \equiv$

```

func canon(s state) state {
  var t state
  t.closed = s.closed
  next := int8(1)
  var mapTo [20]int8
  for c := 0; c < n; c++ {
    v := s.comp[c]
    if v  $\equiv$  black {
      continue
    }
    if mapTo[v]  $\equiv$  0 {
      mapTo[v] = next
      next++
    }
    t.comp[c] = mapTo[v]
  }
  return t
}

func key(s state, i int) string {
  b := make([]byte, 0, n + 6)
  b = strconv.AppendInt(b, int64(i), 36)
  for c := 0; c < n; c++ {
    b = append(b, byte(s.comp[c] + 'a'))
  }
  if s.closed {
    b = append(b, '!')
  }
  return string(b)
}

func has(s state, v int8) bool {
  for c := 0; c < n; c++ {
    if s.comp[c]  $\equiv$  v {
      return true
    }
  }
  return false
}

```

9. Here is the walk. It returns the family of all ways to decide cells $i, i+1, \dots$ given the frontier, and it is memoized on the frontier, which is the whole point: two different histories that leave the same frontier have the same future.

⟨The frontier 7⟩ $\equiv$

```

func build(i int, s state) bdd.Func {
  if  $i \equiv m * n$  {
    ⟨Accept or reject the finished board 10⟩
  }
  s = canon(s)
  k := key(s, i)
  if f, ok := memo[k]; ok {
    return f
  }
  seen ++
  r, c := i / n, i % n
  up, left := black, black
  if r > 0 {
    up = s.comp[c]
  }
  if c > 0 {
    left = s.comp[c - 1]
  }
  res := z.Empty()
  ⟨Paint cell i black 11⟩
  ⟨Leave cell i white 12⟩
  memo[k] = res
  return res
}

```

10. The white region must be connected, so when the walk ends there must be exactly one component still on the frontier—unless a component was completed earlier, in which case it was the only one and nothing white may have followed it.

⟨Accept or reject the finished board 10⟩ $\equiv$

```

var d int
var last int8
for c := 0; c < n; c ++ {
  if v := s.comp[c];  $v \neq \text{black} \wedge v \neq \text{last}$  {
    d ++
    last = v
  }
}
if (s.closed  $\wedge d \equiv 0$ )  $\vee (\neg s.closed \wedge d \equiv 1)$  {
  return z.Unit()
}
return z.Empty()

```

This code is used in section 9.

11. A black cell may not touch a black neighbor. Note which branch carries the ZDD element: the family is a family of *black* sets.

$\langle \text{Paint cell } i \text{ black } 11 \rangle \equiv$

```

if  $\neg(r > 0 \wedge up \equiv black) \wedge \neg(c > 0 \wedge left \equiv black)$  {
   $t := s$ 
   $t.comp[c] = black$ 
  if  $closeCheck(\&t, s, r, c)$  {
     $res = z.Union(res, z.Join(z.Elt(i), build(i + 1, t)))$ 
  }
}

```

This code is used in section 9.

12. A white cell joins the components of its white neighbors, merging them if they were two. If neither neighbor is white it starts a component of its own.

$\langle \text{Leave cell } i \text{ white } 12 \rangle \equiv$

```

if  $\neg s.closed$  {
   $t := s$ 
   $id := up$ 
  if  $id \equiv black$  {
     $id = left$ 
  }
  if  $id \equiv black$  {
     $id = freshID(s)$ 
  } else if  $left \neq black \wedge left \neq id$  {
     $relabel(\&t, left, id)$ 
  }
   $t.comp[c] = id$ 
  if  $closeCheck(\&t, s, r, c)$  {
     $res = z.Union(res, build(i + 1, t))$ 
  }
}

```

This code is used in section 9.

13. Deciding cell i pushes cell $i - n$ off the frontier. If that cell was the last of its component, the component can never grow again, so it has to be the whole white region: nothing white may follow, and a second such completion is a disconnected region.

⟨The frontier 7⟩ +≡

```

func closeCheck(t *state, old state, r, c int) bool {
  if  $r \equiv 0$  {
    return true
  }
   $v := old.comp[c]$ 
  if  $v \equiv black \vee has(*t, v)$  {
    return true
  }
  if t.closed {
    return false
  }
  t.closed = true
  return true
}

func freshID(s state) int8 {
  var used [20]bool
  for  $c := 0; c < n; c++$  {
    used[s.comp[c]] = true
  }
  for  $v := int8(1); ; v++$  {
    if  $\neg used[v]$  {
      return v
    }
  }
}

func relabel(s *state, from, to int8) {
  for  $c := 0; c < n; c++$  {
    if  $s.comp[c] \equiv from$  {
      s.comp[c] = to
    }
  }
}

```


14. And here is the whole computation for one board. The frontier is one row wide, so the rows are taken the short way round. *MaxWeight* with weights of +1 and then −1 reads the largest and smallest number of black cells off the diagram, which is the answer's left-hand matrix.

⟨ The frontier 7 ⟩ +≡

```

func solve(a, b int)(covers *big.Int, lo, hi, fsize, hsize, states int) {
    seen = 0
    m, n = a, b
    if n > m {
        m, n = b, a
    }
    z = bdd.NewZDD(m * n)
    memo = make(map[string]bdd.Func)
    f := build(0, state{ })
    sub := z.Empty()
    for v := 0; v < m * n; v++ {
        sub = z.Union(sub, z.Quotient(f, z.Elt(v)))
    }
    h := z.Diff(f, sub)
    w := make([]int, m * n)
    for i := range w {
        w[i] = 1
    }
    _, hi, _ = z.MaxWeight(h, w)
    for i := range w {
        w[i] = −1
    }
    _, lo, _ = z.MaxWeight(h, w)
    return z.Count(h), −lo, hi, z.Size(f), z.Size(h), seen
}

```

15. Bitboards. The checks that follow—brute force, symmetry, and the two claims the answer makes in prose—all work on explicit boards rather than on diagrams, so they want a small set type. Nine by nine is 81 cells, which is one bit too many for a machine word, so two words it is.

⟨Bitboards 15⟩ ≡

```

type bits [2]uint64
func (b *bits) set(i int) { b[i >> 6] |= 1 << uint(i & 63) }
func (b bits) has(i int) bool { return b[i >> 6] & (1 << uint(i & 63)) ≠ 0 }
func (b bits) and(c bits) bits { return bits{b[0] & c[0], b[1] & c[1]} }
func (b bits) or(c bits) bits { return bits{b[0] | c[0], b[1] | c[1]} }
func (b bits) andn(c bits) bits { return bits{b[0] ⊗ c[0], b[1] ⊗ c[1]} }
func (b bits) zero() bool { return b[0] ≡ 0 ∧ b[1] ≡ 0 }
func (b bits) count()(n int) {
    for -, w := range b {
        for; w ≠ 0; w &= w - 1 {
            n++
        }
    }
    return
}
func (b bits) first() int {
    for k := 0; k < 128; k++ {
        if b.has(k) {
            return k
        }
    }
    return -1
}

```

This code is also defined in sections 16, 17, 18.

This code is used in section 4.

16. A board carries its adjacency lists and the set of all its cells.

⟨ Bitboards 15 ⟩ +≡

```

type geom struct {
    m, n int
    adj []bits
    full bits
}

func newGeom(m, n int) *geom {
    g := &geom{m: m, n: n, adj: make([]bits, m * n)}
    for r := 0; r < m; r++ {
        for c := 0; c < n; c++ {
            i := r * n + c
            g.full.set(i)
            if r > 0 {
                g.adj[i].set(i - n)
            }
            if r + 1 < m {
                g.adj[i].set(i + n)
            }
            if c > 0 {
                g.adj[i].set(i - 1)
            }
            if c + 1 < n {
                g.adj[i].set(i + 1)
            }
        }
    }
    return g
}

```

17. The empty graph is taken to be *not* connected here. That choice matters in exactly one place in the whole exercise—the 1×1 board—and the report beside this program says what happens there.

⟨Bitboards 15⟩ +≡

```

func (g *geom) connected(open bits) bool {
    if open.zero() {
        return false
    }
    seen := bits{}
    seen.set(open.first())
    for {
        grow := seen
        for k := 0; k < g.m * g.n; k++ {
            if seen.has(k) {
                grow = grow.or(g.adj[k].and(open))
            }
        }
        if grow == seen {
            return seen == open
        }
        seen = grow
    }
}

```

18. The three conditions of exercise 439, read off a board directly. This is the definition, transcribed; everything else in this program is an optimization of it, and *brute* below is what checks that the optimizations agree.

⟨Bitboards 15⟩ +≡

```

func (g *geom) cover(b bits) bool {
    for k := 0; k < g.m * g.n; k++ {
        if b.has(k) ^ ¬g.adj[k].and(b).zero() {
            return false // (ii) two black cells touch
        }
    }
    white := g.full.andn(b)
    if ¬g.connected(white) {
        return false // (i)
    }
    for k := 0; k < g.m * g.n; k++ {
        if ¬white.has(k) ∨ ¬g.adj[k].and(b).zero() {
            continue
        }
        w := white
        w[k >> 6] ∘= 1 << uint(k & 63)
        if g.connected(w) {
            return false // (iii) this white cell could be blackened
        }
    }
    return true
}

```

19. Brute force. Small boards can be settled by looking at every independent set, and that is worth doing: it tests the frontier, the memoization, the quotient trick, and the reading of the definition, all at once, against a program that shares none of their machinery.

⟨ Brute force 19 ⟩ $\equiv$

```

func bruteCount(a, b int)(covers, lo, hi int) {
  g := newGeom(a, b)
  N := a * b
  lo = N + 1
  var rec func(k int, black bits)
  rec = func(k int, black bits) {
    if k  $\equiv$  N {
      if g.cover(black) {
        covers ++
        c := black.count()
        lo, hi = min(lo, c), max(hi, c)
      }
      return
    }
    rec(k + 1, black)
    if g.adj[k].and(black).zero() {
      t := black
      t.set(k)
      rec(k + 1, t)
    }
  }
  rec(0, bits{ })
  return
}

```

This code is also defined in section 20.

This code is used in section 4.

20. ⟨ Brute force 19 ⟩ $+ \equiv$

```

func brute(N int) {
  fmt.Printf("\nBrute force against the frontier, for m, n <= %d\n", N)
  bad := 0
  for a := 1; a  $\leq$  N; a ++ {
    for b := a; b  $\leq$  N; b ++ {
      c1, lo1, hi1 := bruteCount(a, b)
      c2, lo2, hi2, -, -, - := solve(a, b)
      ok := c2.IsInt64()  $\wedge$  c2.Int64()  $\equiv$  int64(c1)  $\wedge$  lo1  $\equiv$  lo2  $\wedge$  hi1  $\equiv$  hi2
      if  $\neg$ ok {
        bad ++
        fmt.Printf("%dx%disagree: brute[%d..%d], frontier%s[%d..%d]\n",
          a, b, c1, lo1, hi1, c2, lo2, hi2)
      }
    }
  }
  fmt.Printf("%dboards, %disagreements\n", N * (N + 1)/2, bad)
}

```

21. Symmetry. A cover fixed by a group of board symmetries is a union of orbits of cells, so we enumerate unions of orbits rather than subsets of cells. Orbits that touch themselves can never be black, and an orbit adjacent to one already chosen is out too, which prunes the walk to almost nothing.

⟨Symmetry 21⟩ ≡

```

func (g *geom) orbits(gen []func(int) int) [][]int {
    N := g.m * g.n
    seen := make([]bool, N)
    var out [][]int
    for i := 0; i < N; i++ {
        if seen[i] {
            continue
        }
        o := []int{i}
        seen[i] = true
        for k := 0; k < len(o); k++ {
            for _, f := range gen {
                if j := f(o[k]); ¬seen[j] {
                    seen[j] = true
                    o = append(o, j)
                }
            }
        }
        out = append(out, o)
    }
    return out
}

```

This code is also defined in sections 22, 23, 24, 26, 28.

This code is used in section 4.

22. $\langle \text{Symmetry } 21 \rangle + \equiv$

```

func (g *geom) fixedCovers(gen []func(int) int)(n int, found []bits) {
  orb := g.orbits(gen)
  masks := make([]bits, len(orb))
  ok := make([]bool, len(orb))
  for i, o := range orb {
    var b bits
    for _, k := range o {
      b.set(k)
    }
    masks[i], ok[i] = b, true
    for _, k := range o {
      if ¬g.adj[k].and(b).zero() {
        ok[i] = false
      }
    }
  }
}

var rec func(i int, b bits)
rec = func(i int, b bits) {
  if i ≡ len(orb) {
    if g.cover(b) {
      n++
      found = append(found, b)
    }
    return
  }
  rec(i + 1, b)
  if ok[i] ∧ g.touchesNone(orb[i], b) {
    rec(i + 1, b.or(masks[i]))
  }
}
rec(0, bits{ })
return
}

func (g *geom) touchesNone(orbit []int, b bits) bool {
  for _, k := range orbit {
    if ¬g.adj[k].and(b).zero() {
      return false
    }
  }
  return true
}

```

23. The three symmetries the answer names. t is the transpose, which is a board symmetry only when the board is square.

⟨Symmetry 21⟩ +≡

```
func (g *geom) maps()(h, v, t func(int) int) {
    h = func(i int) int { return (i/g.n) * g.n + (g.n - 1 - i % g.n) }
    v = func(i int) int { return (g.m - 1 - i/g.n) * g.n + i % g.n }
    t = func(i int) int { return (i % g.n) * g.n + i/g.n }
    return
}
```

24. A cover with both mirror symmetries is fixed by the whole Klein group, so it stands alone in its orbit; when the board is square the group has order eight and such a cover is alone only if it is symmetric eight ways, and otherwise comes in a pair. That is what the answer means by counting some covers once and others twice or four times, and the numbers below let one check it.

⟨Symmetry 21⟩ +≡

```
func symmetry(N int) {
    fmt.Println("\nCovers with both horizontal and vertical symmetry")
    for a := 1; a ≤ N; a++ {
        for b := 1; b ≤ N; b++ {
            g := newGeom(a, b)
            h, v, _ := g.maps()
            k, _ := g.fixedCovers([]func(int) int { h, v })
            fmt.Printf("%6d", k)
        }
        fmt.Println()
    }
    ⟨Report the square board's finer symmetries 25⟩
}
```

25. ⟨Report the square board's finer symmetries 25⟩ ≡

```
fmt.Printf("\n%3s%10s%18s%20s\n", "n", "8-fold", "90deg(not 8)",
    "both diagonals(not 8)")
for a := 1; a ≤ N; a++ {
    g := newGeom(a, a)
    h, v, t := g.maps()
    eight, _ := g.fixedCovers([]func(int) int { h, v, t })
    rot, _ := g.fixedCovers([]func(int) int { func(i int) int { return h(t(i)) } })
    diag, _ := g.fixedCovers([]func(int) int { t, func(i int) int { return h(v(t(i))) } })
    fmt.Printf("%3d%10d%18d%20d\n", a, eight, (rot - eight)/2, (diag - eight)/2)
}
```

This code is used in section 24.

26. The claims in prose. Two of the answer's assertions are sentences rather than numbers. The first: fourfold horizontal and vertical symmetry is impossible when m and n are both even, *because it forces at least 12 white cells near the center*.

The reason is worth spelling out, because the program checks it rather than merely checking the conclusion. With both mirrors and both dimensions even there is no middle row and no middle column, so the four cells of the central 2×2 block form one orbit—and two of them are adjacent, so the orbit cannot be black. Each of the eight cells around that block likewise lies in an orbit containing its own mirror image next door. So twelve cells are white, and the middle four of them have no black neighbor and are not cut vertices: the cover is not maximal.

⟨Symmetry 21⟩ $\equiv$

```
func claims(N int) {
  fmt.Println("\nAre the twelve cells at the centre always white? (m, n both even)")
  for a := 2; a ≤ N; a += 2 {
    for b := a; b ≤ N; b += 2 {
      g := newGeom(a, b)
      h, v, _ := g.maps()
      ⟨Mark the central twelve cells 27⟩
      fmt.Printf(" %dx%d: %v%s\n", a, b, g.allWhite([func(int) int { h, v }, centre), note)
    }
  }
}
```

27. The twelve are the central block and its neighbors. On a $2 \times n$ board some of them fall outside, and the claim is then about fewer cells.

⟨Mark the central twelve cells 27⟩ $\equiv$

```
var centre bits
note := ""
for _, d := range [[2]int { {-1, -1}, {-1, 0}, {0, -1}, {0, 0},
  {-2, -1}, {-2, 0}, {1, -1}, {1, 0},
  {-1, -2}, {0, -2}, {-1, 1}, {0, 1} }]{
  r, c := a/2 + d[0], b/2 + d[1]
  if r < 0 ∨ r ≥ a ∨ c < 0 ∨ c ≥ b {
    note = " (the board is too small to hold all twelve)"
    continue
  }
  centre.set(r * b + c)
}
```

This code is used in section 26.

28. $\langle \text{Symmetry } 21 \rangle + \equiv$

```

func (g *geom) allWhite(gen []func(int) int, cells bits) bool {
  orb := g.orbits(gen)
  ok := true
  var rec func(i int, b bits)
  rec = func(i int, b bits) {
    if  $\neg$ ok {
      return
    }
    if i  $\equiv$  len(orb) {
      if g.connected(g.full.andn(b))  $\wedge$   $\neg$ b.and(cells).zero() {
        ok = false
      }
      return
    }
    rec(i + 1, b)
    var mask bits
    for  $\_$ , k := range orb[i] {
      mask.set(k)
    }
    if g.touchesNone(orb[i], b.or(mask)) {
      rec(i + 1, b.or(mask))
    }
  }
  rec(0, bits{ })
  return ok
}

```

29. The second claim is a recurrence: the number X_n of $2 \times n$ hitori covers satisfies $X_n = 2X_{n-2} + 2X_{n-3}$ and grows as $\Theta(r^n)$ with $r \approx 1.76929$. That r should be the real root of $x^3 = 2x + 2$, and we check both the recurrence, well past the nine columns the exercise asks for, and the root.

⟨Reporting 29⟩ $\equiv$

```

func twoByN() {
  fmt.Println("\n2x_n covers and the recurrence X(n) = 2X(n-2) + 2X(n-3)")
  x := make([]int64, 25)
  for b := 1; b ≤ 24; b++ {
    c, -, -, -, - := solve(2, b)
    x[b] = c.Int64()
  }
  bad := 0
  for b := 4; b ≤ 24; b++ {
    if x[b] ≠ 2 * x[b-2] + 2 * x[b-3] {
      bad++
      fmt.Printf("n=%d: d, but 2*d + 2*d = d\n", b, x[b], x[b-2], x[b-3],
        2 * x[b-2] + 2 * x[b-3])
    }
  }
  fmt.Printf("X(1..12) = v\n", x[1:13])
  fmt.Printf("recurrence holds for n = 4..24 with %d exceptions\n", bad)
  ⟨Find the growth rate 30⟩
}

```

This code is also defined in sections 31, 33.

This code is used in section 4.

30. Newton's method on $x^3 - 2x - 2$ converges in a moment.

⟨Find the growth rate 30⟩ $\equiv$

```

r := 1.7
for i := 0; i < 100; i++ {
  r -= (r * r * r - 2 * r - 2) / (3 * r * r - 2)
}
fmt.Printf("root of x^3 = 2x + 2 is %.9f; X(24)/X(23) = %.6f\n",
  r, float64(x[24])/float64(x[23]))
_ = math.Abs

```

This code is used in section 29.

31. Reporting. The main table. Besides the totals and the range of black cells—the answer’s two matrices—we print what the diagrams cost, because the answer quotes three sizes for the 9×9 case and they are worth comparing.

⟨Reporting 29⟩ $\equiv$

```

func counts(N int) {
  cnt := make([][]string, N + 1)
  rng := make([][]string, N + 1)
  for i := range cnt {
    cnt[i] = make([]string, N + 1)
    rng[i] = make([]string, N + 1)
  }
  for a := 1; a ≤ N; a++ {
    for b := a; b ≤ N; b++ {
      t0 := time.Now()
      c, lo, hi, fs, hs, st := solve(a, b)
      cnt[a][b], cnt[b][a] = c.String(), c.String()
      r := fmt.Sprintf("%d..%d", lo, hi)
      rng[a][b], rng[b][a] = r, r
      if a ≡ b ∧ a ≡ N {
        fmt.Printf("%dx%d: %s covers; %d frontier states, " +
          "ZDD %d for the first family, %d for the maximal one, in %v\n",
            a, b, c, st, fs, hs, time.Since(t0).Round(time.Millisecond))
      }
    }
  }
  ⟨Print the two matrices 32⟩
}

```

32. ⟨Print the two matrices 32⟩ $\equiv$

```

fmt.Println("\nHitori covers of PmxPn")
for a := 1; a ≤ N; a++ {
  for b := 1; b ≤ N; b++ {
    fmt.Printf("%13s", cnt[a][b])
  }
  fmt.Println()
}
fmt.Println("\nBlack cells [fewest..most]")
for a := 1; a ≤ N; a++ {
  for b := 1; b ≤ N; b++ {
    fmt.Printf("%10s", rng[a][b])
  }
  fmt.Println()
}

```

This code is used in section 31.

33. Finally the gallery. The answer's Fig. A-6 shows some of the winners of its beauty contest, and the eight-way symmetric covers are the rarest of them: one for $n = 1$, one for $n = 3$, two for $n = 5$, two for $n = 7$, and eleven for $n = 9$. Here they are, drawn with # for black.

⟨Reporting 29⟩ +≡

```

func gallery(N int) {
  fmt.Println("\\nThe 8-fold symmetric covers")
  for a := 1; a ≤ N; a += 2 {
    g := newGeom(a, a)
    h, v, t := g.maps()
    _, found := g.fixedCovers([]func(int) int { h, v, t })
    fmt.Printf("n=%d: %d of them\\n", a, len(found))
    for _, b := range found {
      ⟨Draw one cover 34⟩
    }
  }
}

```

34. ⟨Draw one cover 34⟩ ≡

```

for r := 0; r < a; r++ {
  line := ""
  for c := 0; c < a; c++ {
    if b.has(r * a + c) {
      line += "#"
    } else {
      line += "."
    }
  }
  fmt.Println(line)
}
fmt.Println()

```

This code is used in section 33.

35. Index.*a*: [14](#), [19](#), [20](#), [24](#), [25](#), [26](#), [27](#), [31](#), [32](#), [33](#), [34](#).*Abs*: [30](#).*adj*: [16](#), [17](#), [18](#), [19](#), [22](#).*allWhite*: [26](#), [28](#).*and*: [15](#), [17](#), [18](#), [19](#), [22](#), [28](#).*andn*: [15](#), [18](#), [28](#).*append*: [8](#), [21](#), [22](#).*AppendInt*: [8](#).*b*: [8](#), [14](#), [15](#), [18](#), [19](#), [20](#), [22](#), [24](#), [26](#), [27](#), [28](#), [29](#), [31](#), [32](#), [33](#), [34](#).*bad*: [20](#), [29](#).**bdd**: [3](#), [7](#), [9](#), [14](#).**big**: [14](#).**bits**: [15](#), [16](#), [17](#), [18](#), [19](#), [22](#), [27](#), [28](#).*black*: [7](#), [8](#), [9](#), [10](#), [11](#), [12](#), [13](#), [19](#).*bool*: [7](#), [8](#), [13](#), [15](#), [17](#), [18](#), [21](#), [22](#), [28](#).*brute*: [6](#), [18](#), [20](#).*bruteCount*: [19](#), [20](#).*build*: [9](#), [11](#), [12](#), [14](#).*byte*: [8](#).*c*: [7](#), [8](#), [9](#), [10](#), [11](#), [12](#), [13](#), [15](#), [16](#), [19](#), [27](#), [29](#), [31](#), [34](#).*c1*: [20](#).*c2*: [20](#).*canon*: [8](#), [9](#).*cells*: [28](#).*centre*: [26](#), [27](#).*claims*: [6](#), [26](#).*closeCheck*: [11](#), [12](#), [13](#).*closed*: [7](#), [8](#), [10](#), [12](#), [13](#).*cnt*: [31](#), [32](#).*comp*: [7](#), [8](#), [9](#), [10](#), [11](#), [12](#), [13](#).*connected*: [17](#), [18](#), [28](#).*Count*: [14](#).*count*: [15](#), [19](#).*counts*: [6](#), [31](#).*cover*: [18](#), [19](#), [22](#).*covers*: [14](#), [19](#).*d*: [10](#), [27](#).*diag*: [25](#).*Diff*: [14](#).*eight*: [25](#).*Elt*: [11](#), [14](#).*Empty*: [9](#), [10](#), [14](#).*f*: [9](#), [14](#), [21](#).*first*: [15](#), [17](#).*fixedCovers*: [22](#), [24](#), [25](#), [33](#).*flag*: [5](#).*float64*: [30](#).*fmt*: [20](#), [24](#), [25](#), [26](#), [29](#), [30](#), [31](#), [32](#), [33](#), [34](#).*found*: [22](#), [33](#).*freshID*: [12](#), [13](#).*from*: [13](#).*fs*: [31](#).*fsize*: [14](#).*full*: [16](#), [18](#), [28](#).**Func**: [7](#), [9](#), [14](#).*g*: [16](#), [17](#), [18](#), [19](#), [21](#), [22](#), [23](#), [24](#), [25](#), [26](#), [28](#), [33](#).*gallery*: [6](#), [33](#).*gen*: [21](#), [22](#), [28](#).**geom**: [16](#), [17](#), [18](#), [21](#), [22](#), [23](#), [28](#).*grow*: [17](#).*h*: [14](#), [23](#), [24](#), [25](#), [26](#), [33](#).*has*: [8](#), [13](#), [15](#), [17](#), [18](#), [34](#).*hi*: [14](#), [19](#), [31](#).*hi1*: [20](#).*hi2*: [20](#).*hs*: [31](#).*hsize*: [14](#).*i*: [8](#), [9](#), [11](#), [12](#), [14](#), [15](#), [16](#), [21](#), [22](#), [23](#), [25](#), [28](#), [30](#), [31](#).*id*: [12](#).*int*: [7](#), [8](#), [9](#), [10](#), [13](#), [14](#), [15](#), [16](#), [19](#), [20](#), [21](#), [22](#), [23](#), [24](#), [25](#), [26](#), [27](#), [28](#), [31](#), [33](#).**Int**: [5](#), [14](#).*Int64*: [20](#), [29](#).*int64*: [8](#), [20](#), [29](#).*int8*: [7](#), [8](#), [10](#), [13](#).*IsInt64*: [20](#).*j*: [21](#).*Join*: [11](#).*k*: [9](#), [15](#), [17](#), [18](#), [19](#), [21](#), [22](#), [24](#), [28](#).*key*: [8](#), [9](#).*last*: [10](#).*left*: [9](#), [11](#), [12](#).*len*: [21](#), [22](#), [28](#), [33](#).*line*: [34](#).*lo*: [14](#), [19](#), [31](#).*lo1*: [20](#).*lo2*: [20](#).*m*: [7](#), [9](#), [14](#), [16](#), [17](#), [18](#), [21](#), [23](#).*main*: [4](#).*make*: [8](#), [14](#), [16](#), [21](#), [22](#), [29](#), [31](#).*maps*: [23](#), [24](#), [25](#), [26](#), [33](#).*mapTo*: [8](#).*mask*: [28](#).

masks: [22](#).
math: [30](#).
max: [19](#).
MaxWeight: [14](#).
memo: [7](#), [9](#), [14](#).
Millisecond: [31](#).
min: [6](#), [19](#).
mode: [5](#), [6](#).
N: [6](#), [19](#), [20](#), [21](#), [24](#), [25](#), [26](#), [31](#), [32](#), [33](#).
n: [7](#), [8](#), [9](#), [10](#), [13](#), [14](#), [15](#), [16](#), [17](#), [18](#), [21](#), [22](#), [23](#).
newGeom: [16](#), [19](#), [24](#), [25](#), [26](#), [33](#).
NewZDD: [14](#).
next: [8](#).
note: [26](#), [27](#).
Now: [31](#).
o: [21](#), [22](#).
ok: [9](#), [20](#), [22](#), [28](#).
old: [13](#).
open: [17](#).
or: [15](#), [17](#), [22](#), [28](#).
orb: [22](#), [28](#).
orbit: [22](#).
orbits: [21](#), [22](#), [28](#).
out: [21](#).
Parse: [5](#).
Printf: [20](#), [24](#), [25](#), [26](#), [29](#), [30](#), [31](#), [32](#), [33](#).
Println: [24](#), [26](#), [29](#), [32](#), [33](#), [34](#).
Quotient: [14](#).
r: [9](#), [11](#), [12](#), [13](#), [16](#), [27](#), [30](#), [31](#), [34](#).
rec: [19](#), [22](#), [28](#).
relabel: [12](#), [13](#).
res: [9](#), [11](#), [12](#).
rng: [31](#), [32](#).
rot: [25](#).
Round: [31](#).
s: [8](#), [9](#), [10](#), [11](#), [12](#), [13](#).
seen: [7](#), [9](#), [14](#), [17](#), [21](#).
set: [15](#), [16](#), [17](#), [19](#), [22](#), [27](#), [28](#).
Since: [31](#).
Size: [14](#).
size: [5](#), [6](#).
solve: [14](#), [20](#), [29](#), [31](#).
Sprintf: [31](#).
st: [31](#).
state: [7](#), [8](#), [9](#), [13](#), [14](#).
states: [14](#).
strconv: [8](#).
String: [5](#), [31](#).
string: [7](#), [8](#), [14](#), [31](#).
sub: [14](#).
symmetry: [6](#), [24](#).
t: [8](#), [11](#), [12](#), [13](#), [19](#), [23](#), [25](#), [33](#).
t0: [31](#).
time: [31](#).
to: [13](#).
touchesNone: [22](#), [28](#).
twoByN: [6](#), [29](#).
uint: [15](#), [18](#).
uint64: [15](#).
Union: [11](#), [12](#), [14](#).
Unit: [10](#).
up: [9](#), [11](#), [12](#).
used: [13](#).
v: [8](#), [10](#), [13](#), [14](#), [23](#), [24](#), [25](#), [26](#), [33](#).
w: [14](#), [15](#), [18](#).
white: [18](#).
x: [29](#), [30](#).
z: [7](#), [9](#), [10](#), [11](#), [12](#), [14](#).
ZDD: [7](#).
zero: [15](#), [17](#), [18](#), [19](#), [22](#), [28](#).

- ⟨Accept or reject the finished board 10⟩ Used in section 9
- ⟨Bitboards 15⟩ Used in section 4
- ⟨Brute force 19⟩ Used in section 4
- ⟨Do what the mode asks 6⟩ Used in section 4
- ⟨Draw one cover 34⟩ Used in section 33
- ⟨Find the growth rate 30⟩ Used in section 29
- ⟨Leave cell i white 12⟩ Used in section 9
- ⟨Mark the central twelve cells 27⟩ Used in section 26
- ⟨Paint cell i black 11⟩ Used in section 9
- ⟨Print the two matrices 32⟩ Used in section 31
- ⟨Read the command line 5⟩ Used in section 4
- ⟨Report the square board's finer symmetries 25⟩ Used in section 24
- ⟨Reporting 29⟩ Used in section 4
- ⟨Symmetry 21⟩ Used in section 4
- ⟨The frontier 7⟩ Used in section 4

Hitori covers

	Section	Page
Introduction	1	1
The frontier	7	4
Bitboards	15	10
Brute force	19	13
Symmetry	21	14
The claims in prose	26	17
Reporting	31	20
Index	35	22